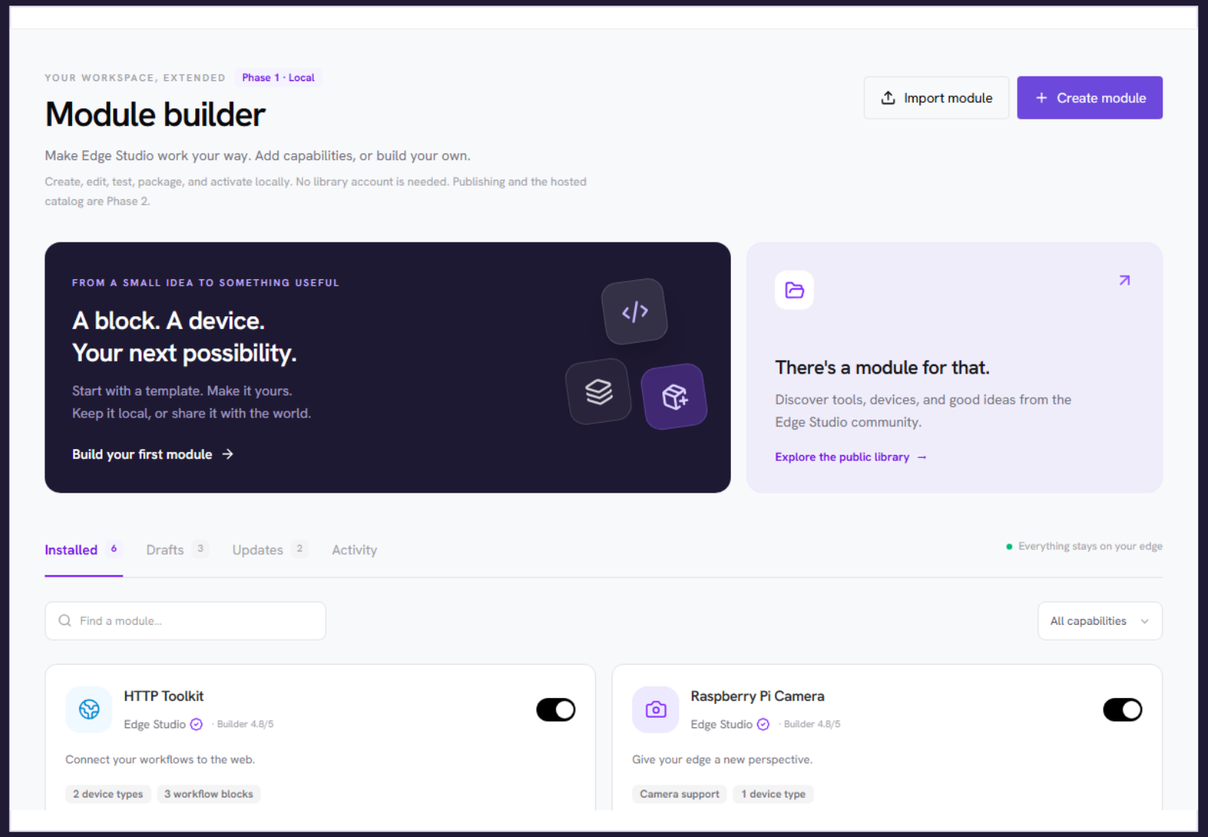
Module Builder

From an unsupported sensor to a shareable capability

A local creation journey

A separate hosted ecosystem



Frontend concept screenshots · Implementation not started

One package. Two delivery phases.

The local builder delivers value before the public library exists.

PHASE 1 · LOCAL

Create and use

Template → own AI harness → edit → preview/test → validate/package → back up → activate → export/import

PHASE 2 · HOSTED

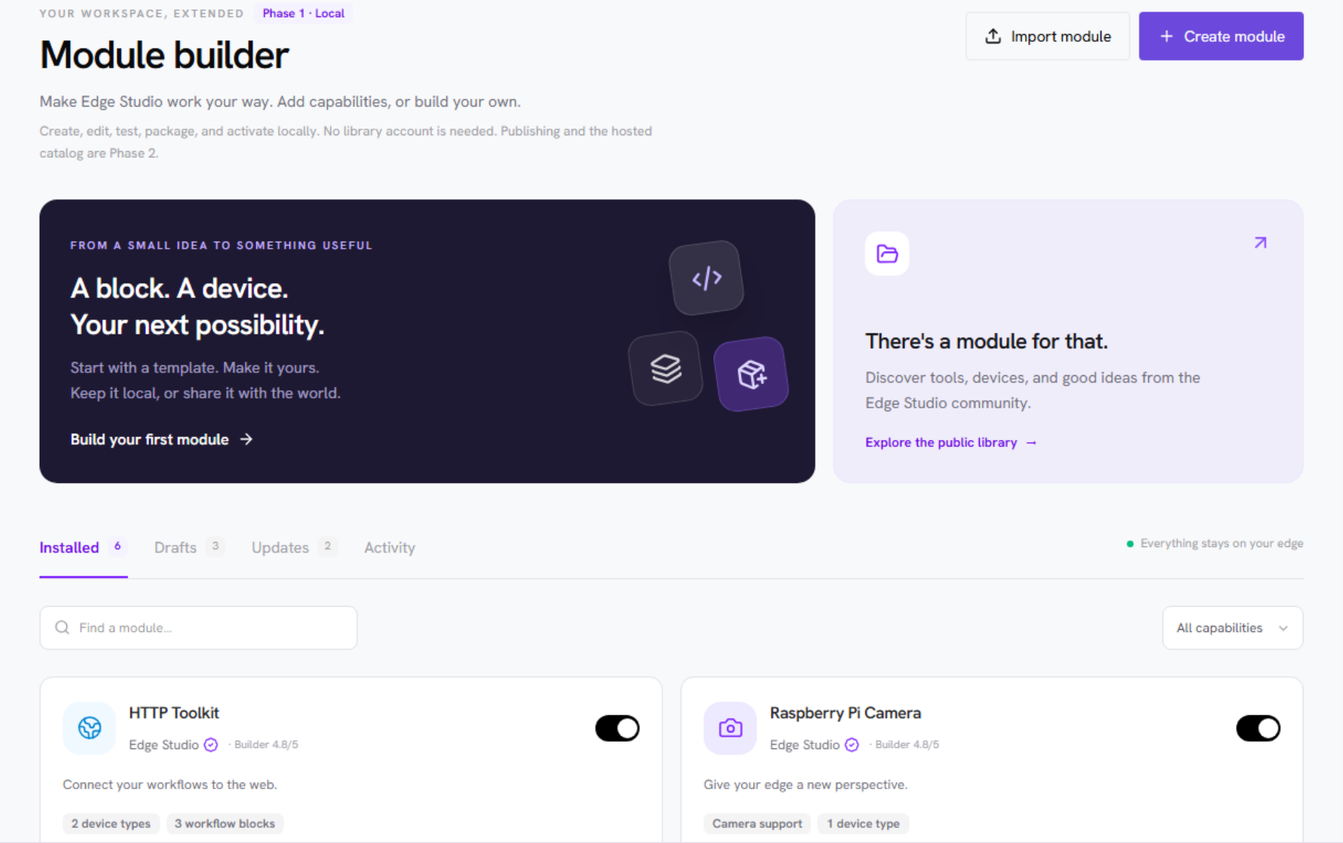
Share and discover

Prepare → automated checks → publish → community inspection/reputation → discover → install

No module CLI. No built-in AI generation service. Local use does not require publication.

01 / Start in your local workspace

PHASE 1 · DISCOVER YOUR LOCAL CAPABILITIES



Captured from the demo frontend; explanatory text is editable.

Focus: Installed modules and local drafts

One place to manage

See installed capabilities, drafts, manual updates and recent activity.

Choose your next step

Create from a template or import a module from another instance.

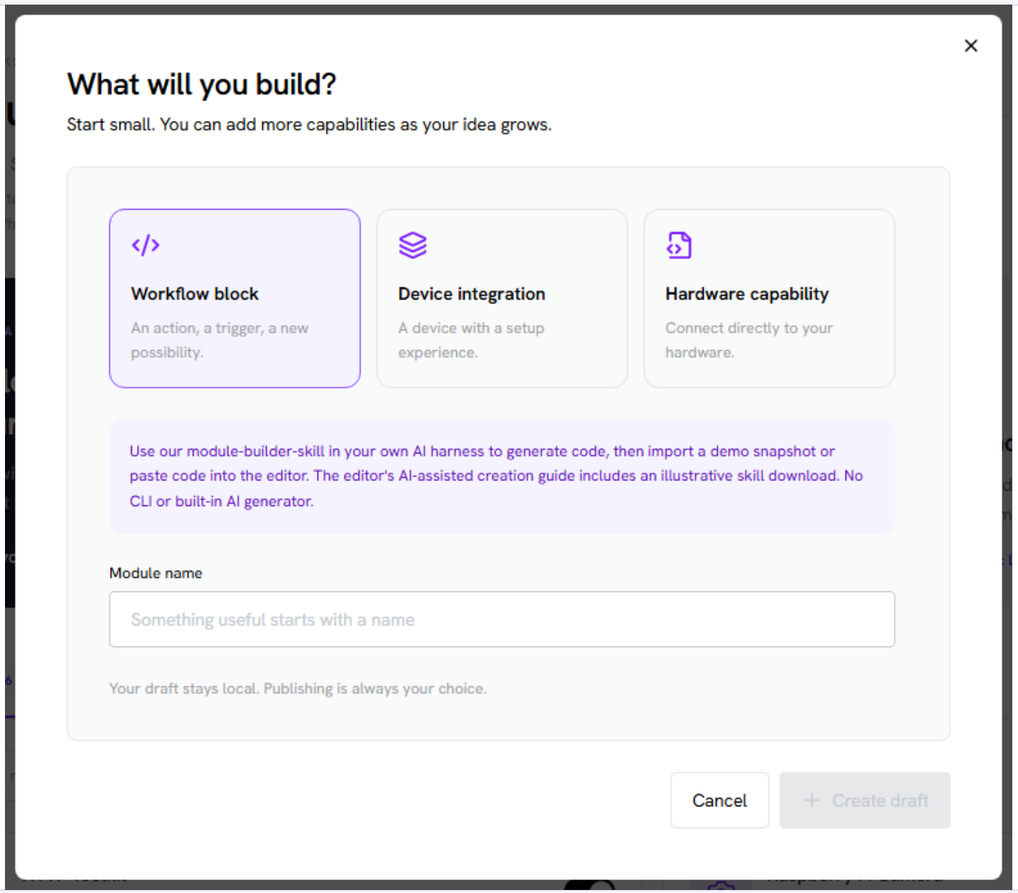
Local first

Creation and local activation need no publisher account or hosted library.

A missing capability can become a local project instead of a core-team feature request.

02 / Choose what the module contributes

PHASE 1 · CREATE



Captured from the demo frontend; explanatory text is editable.

Focus: A useful starting point

Three contribution surfaces

Start with a workflow block, device integration or hardware capability.

Reduce boilerplate

Templates provide an example manifest, source, configuration schema and documentation.

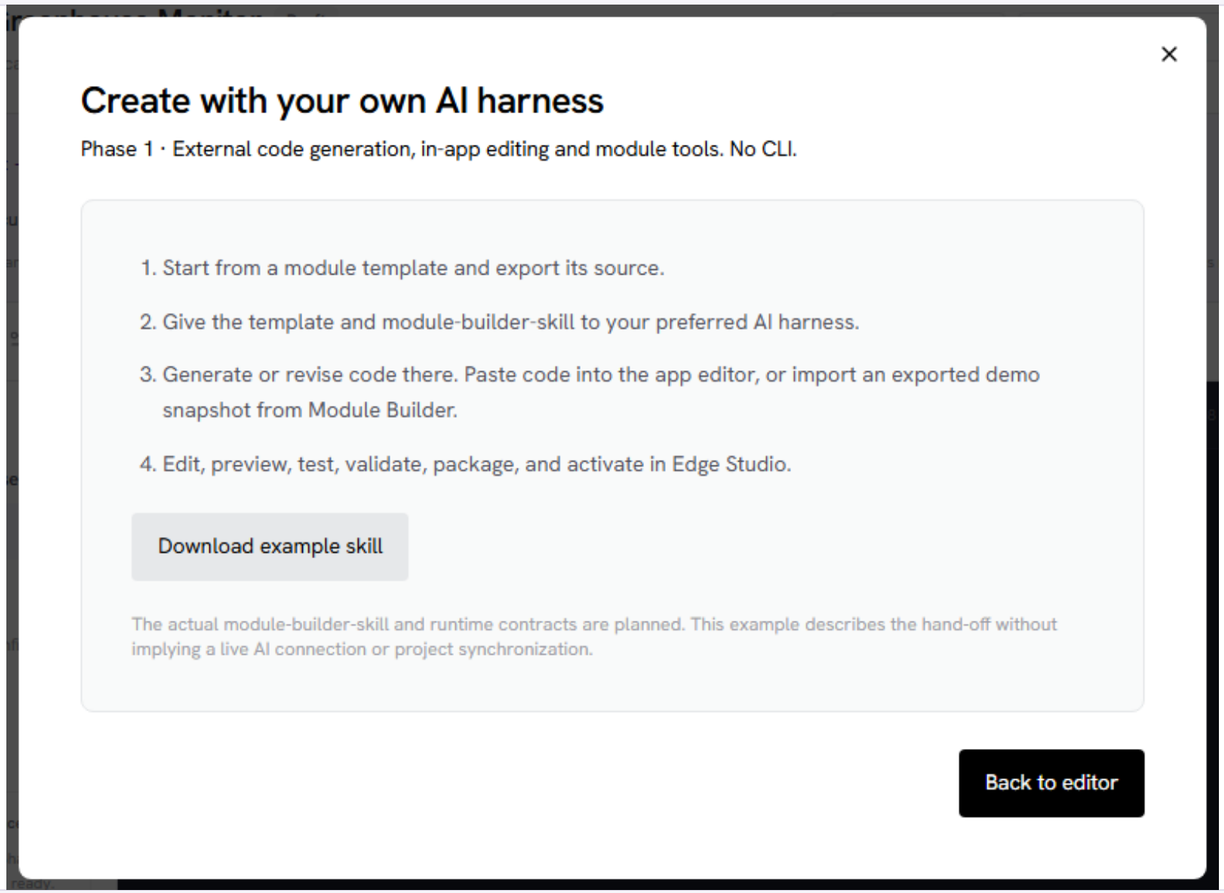
Keep the contract clear

Modules integrate through defined contracts; arbitrary code is not automatically a device.

Template authors and driver developers can use the same package model.

03 / Generate in your own AI harness

PHASE 1 · AUTHOR WITH YOUR TOOLS



Captured from the demo frontend; explanatory text is editable.

Focus: The module-builder-skill hand-off

Bring the contract to the AI

Use our planned skill and template in the harness you already work with.

Bring code back to the app

Paste or import generated source, then edit and check it locally.

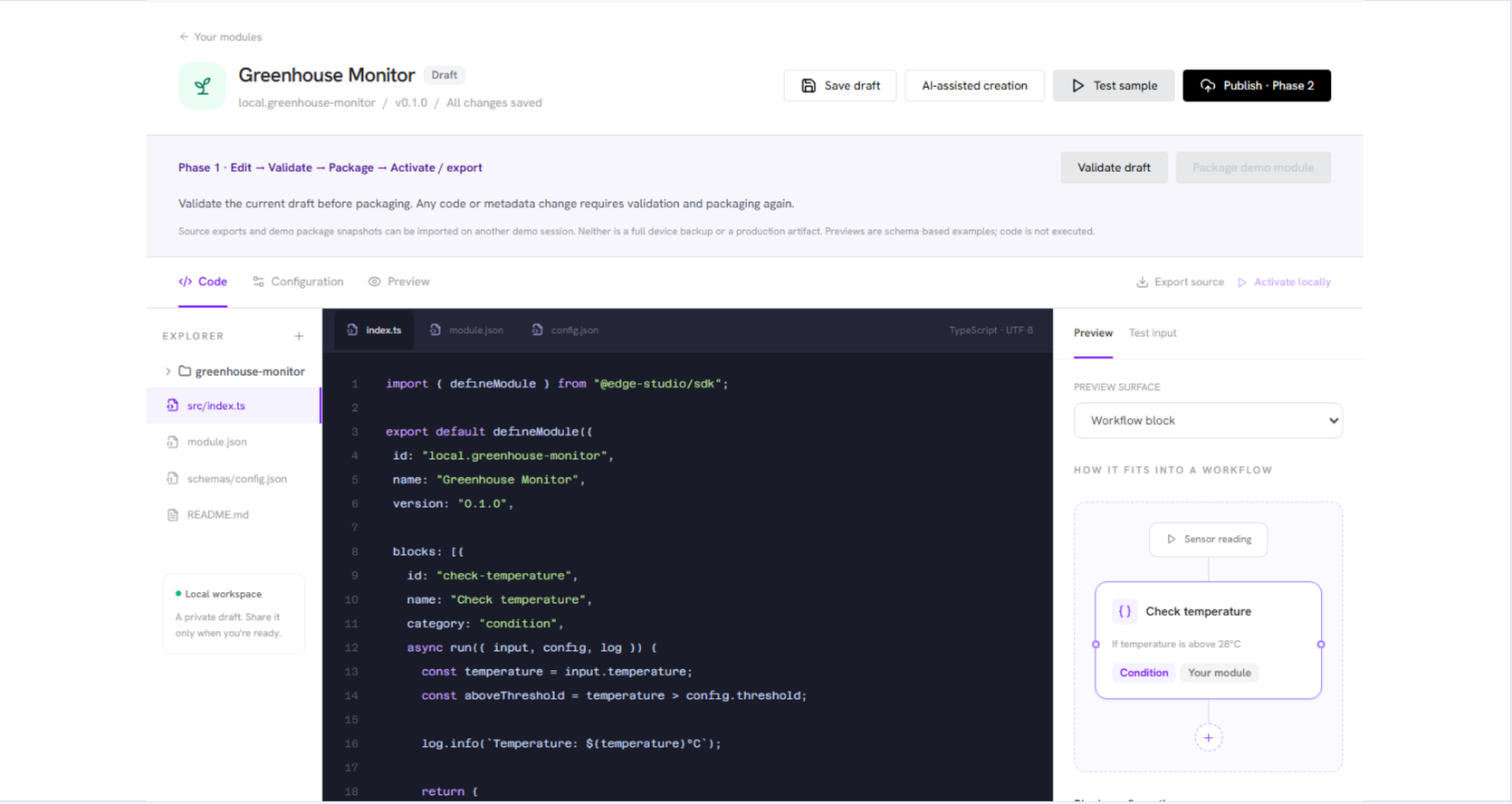
A deliberate product boundary

We provide the module journey without building an AI coding harness or CLI.

The downloadable skill in this screenshot is illustrative, not a released developer contract.

04 / Edit code with the outcome in view

PHASE 1 · EDIT



Captured from the demo frontend; explanatory text is editable.

Focus: Source, configuration and preview

Code editing stays in the app

Inspect files and make changes after the external generation step.

Configuration is a schema

Forms, guides and wizard steps use common rendering contracts.

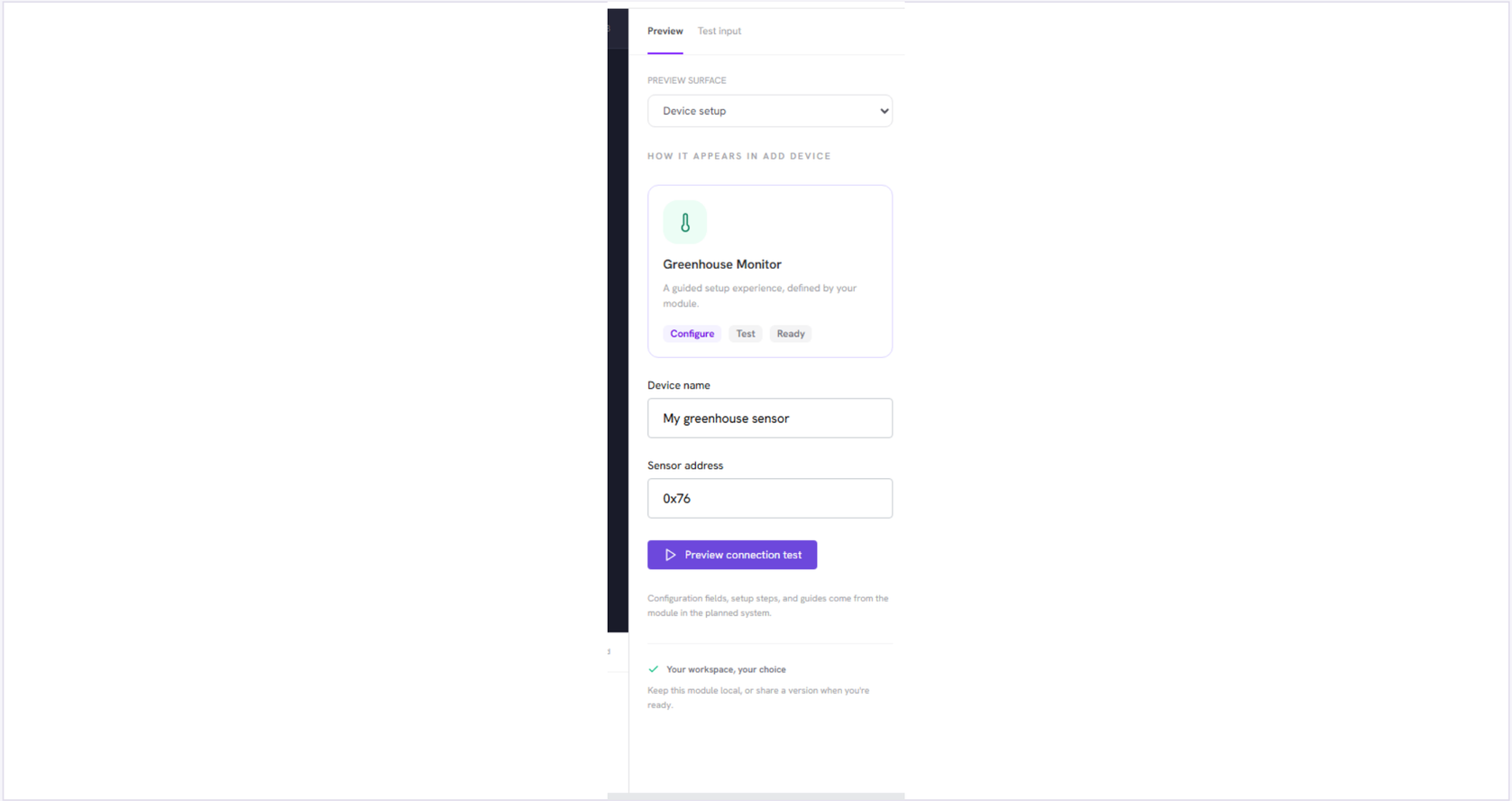
Scope stays focused

Arbitrary custom views and block editors wait for a concrete required example.

The code editor supports module work; it is not a commitment to a full IDE.

05 / Preview device setup

PHASE 1 · PREVIEW THE USER EXPERIENCE



Captured from the demo frontend; explanatory text is editable.

Focus: A schema becomes a setup experience

Preview before activation

See the fields and guidance an operator would encounter when adding a device.

Keep the UI consistent

Modules contribute configuration and instructions to the existing app experience.

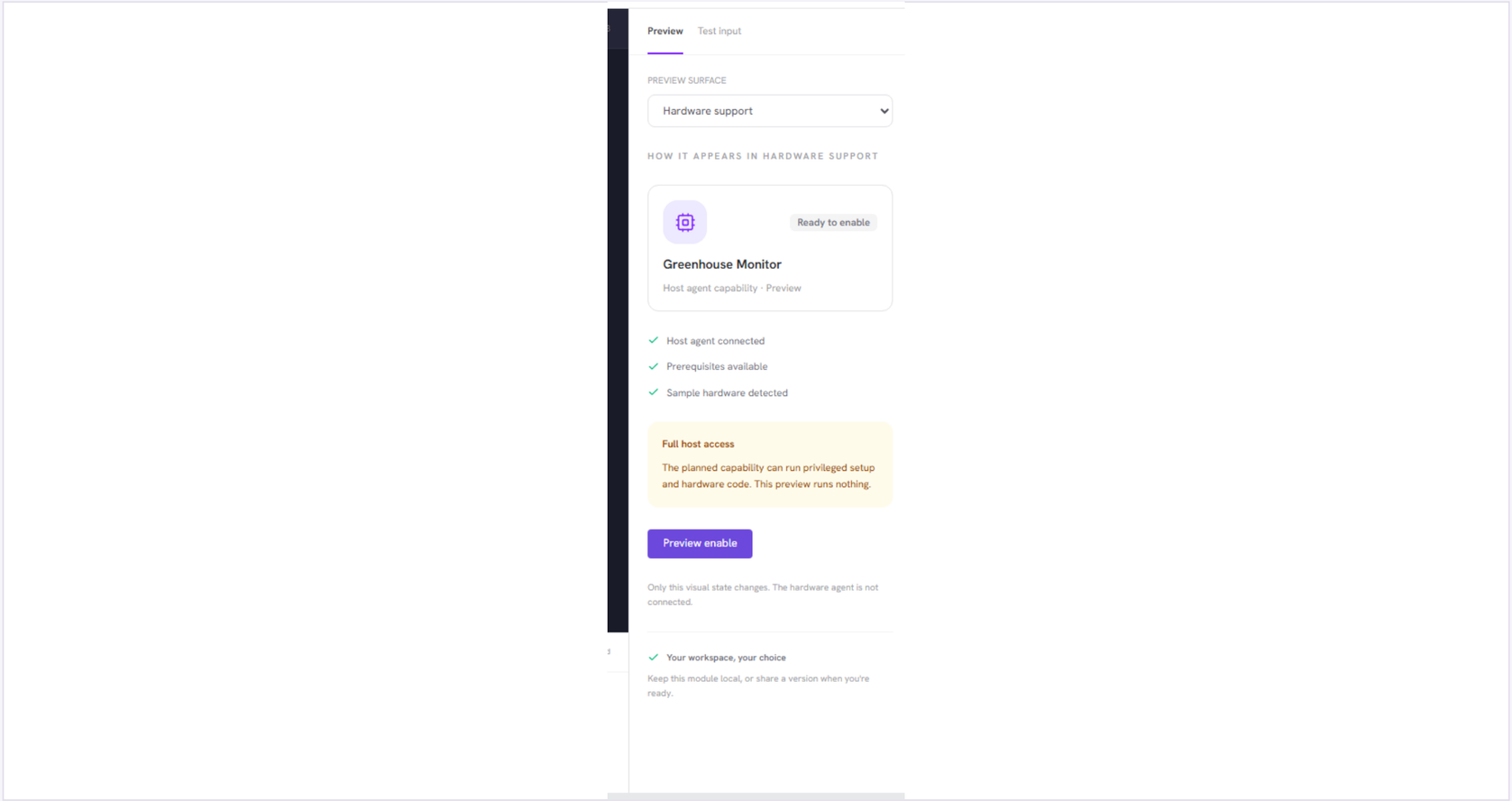
Separate preview from execution

A visual preview is not proof that a sensor, driver or host action works.

The demo renders a fixed example; real hardware acceptance belongs in the implementation.

06 / Preview a hardware capability

PHASE 1 · DECLARE HOST BEHAVIOR



Captured from the demo frontend; explanatory text is editable.

Focus: Hardware support is a module contribution

Declare lifecycle actions

Hardware packages need setup, enable/disable, status and cleanup behavior.

Full host access is intentional

A privileged extension is trusted host software, not sandboxed code.

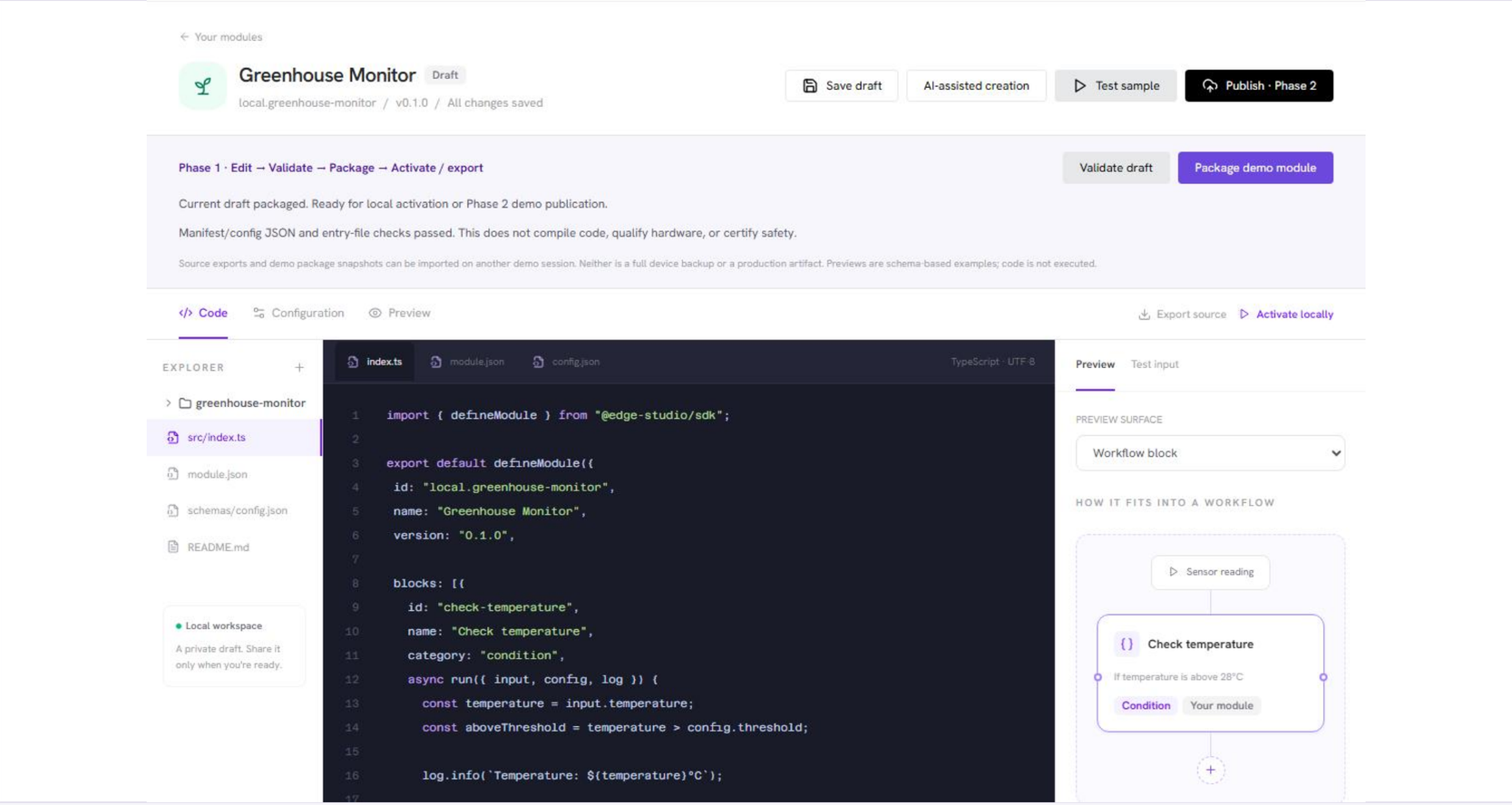
Show the operational outcome

Explain prerequisites, failures, reboot needs and what uninstall can leave behind.

Process supervision manages operation; it does not contain malicious root code.

07 / Test, validate and package a revision

PHASE 1 · PREPARE A RELEASE



Captured from the demo frontend; explanatory text is editable.

Focus: A visible preparation sequence

Try sample inputs

Preview representative outputs and inspect logs before activation.

Validate package structure

Check metadata, configuration and entry points; deeper implementation checks follow the runtime contract.

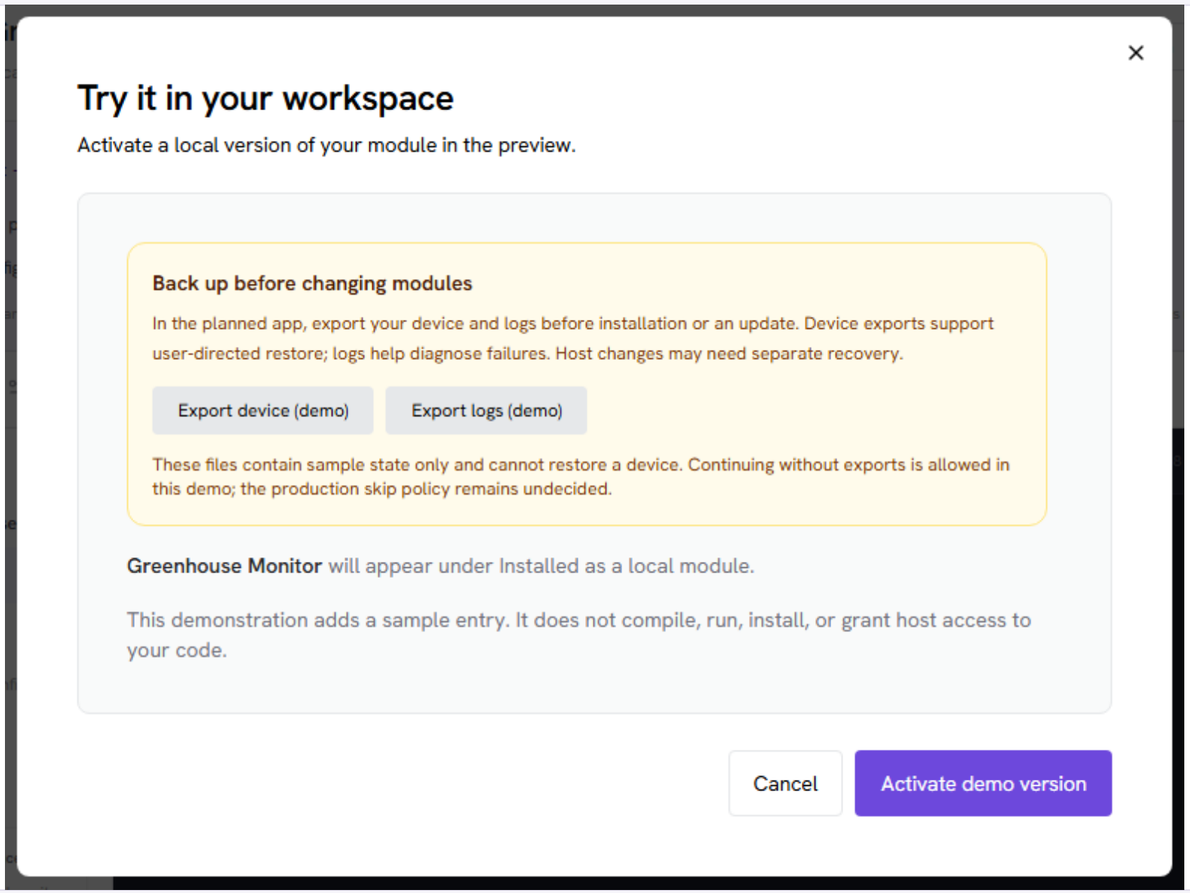
Package the exact revision

Code or metadata changes invalidate preparation and require a fresh package.

The demo performs JSON/entry checks and exports a snapshot; it never compiles or executes code.

08 / Back up before changing the device

PHASE 1 · USER-DIRECTED RECOVERY



Captured from the demo frontend; explanatory text is editable.

Focus: Device export plus diagnostic logs

Prompt before execution

Offer device and log exports before install/update hooks or local activation.

Restore managed state

The planned app-wide device backup must have a tested restore path.

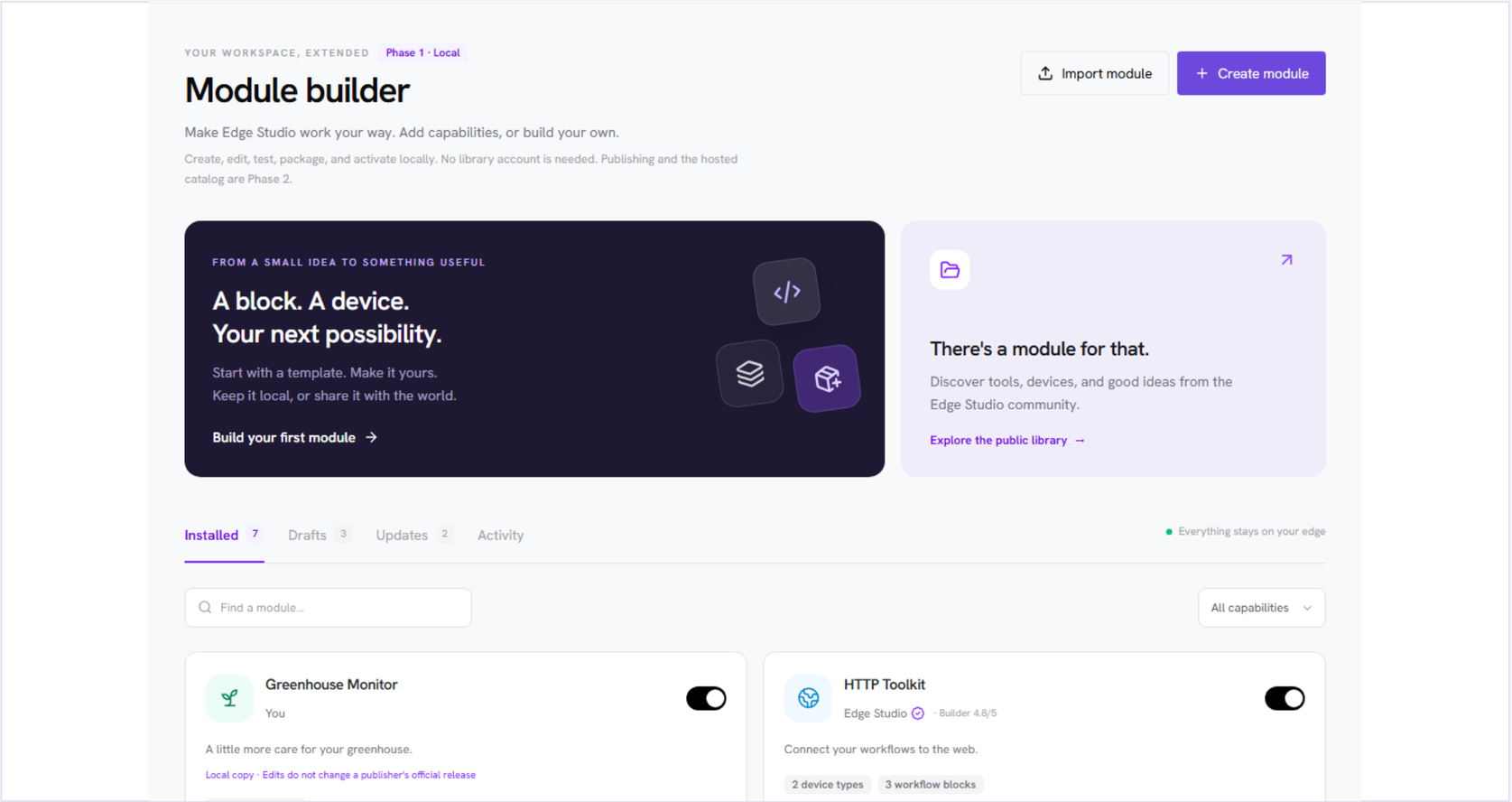
Be precise about coverage

Logs are diagnostic. App backups do not universally undo OS, firmware or hardware changes.

These demo downloads are explicitly non-restorable samples; the production skip policy remains open.

09 / Activate locally and retain control

PHASE 1 · USE AND MAINTAIN



Captured from the demo frontend; explanatory text is editable.

Focus: Installed is different from active

Activate explicitly

A library installation first adds an inactive package; the operator chooses when to activate.

Keep lifecycle actions manual

Enable, disable and update through clear user actions.

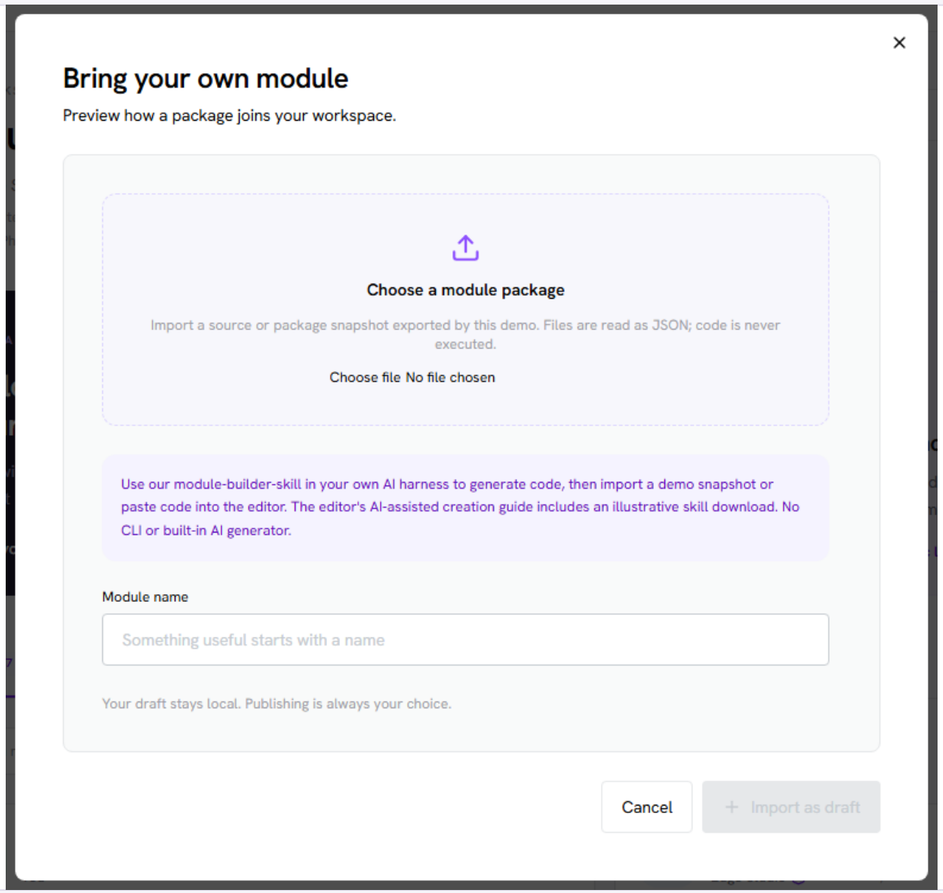
Preserve useful state

Removal retains device/workflow configuration and history unless deletion is explicitly requested.

The local builder remains usable with no production library connection.

10 / Share directly between instances

PHASE 1 · EXPORT / IMPORT



Captured from the demo frontend; explanatory text is editable.

Focus: A package is not a device backup

Export source or a package

Prepare a shareable module without exporting configured credentials or device history.

Import on another instance

Prove author independence without publishing or changing core source.

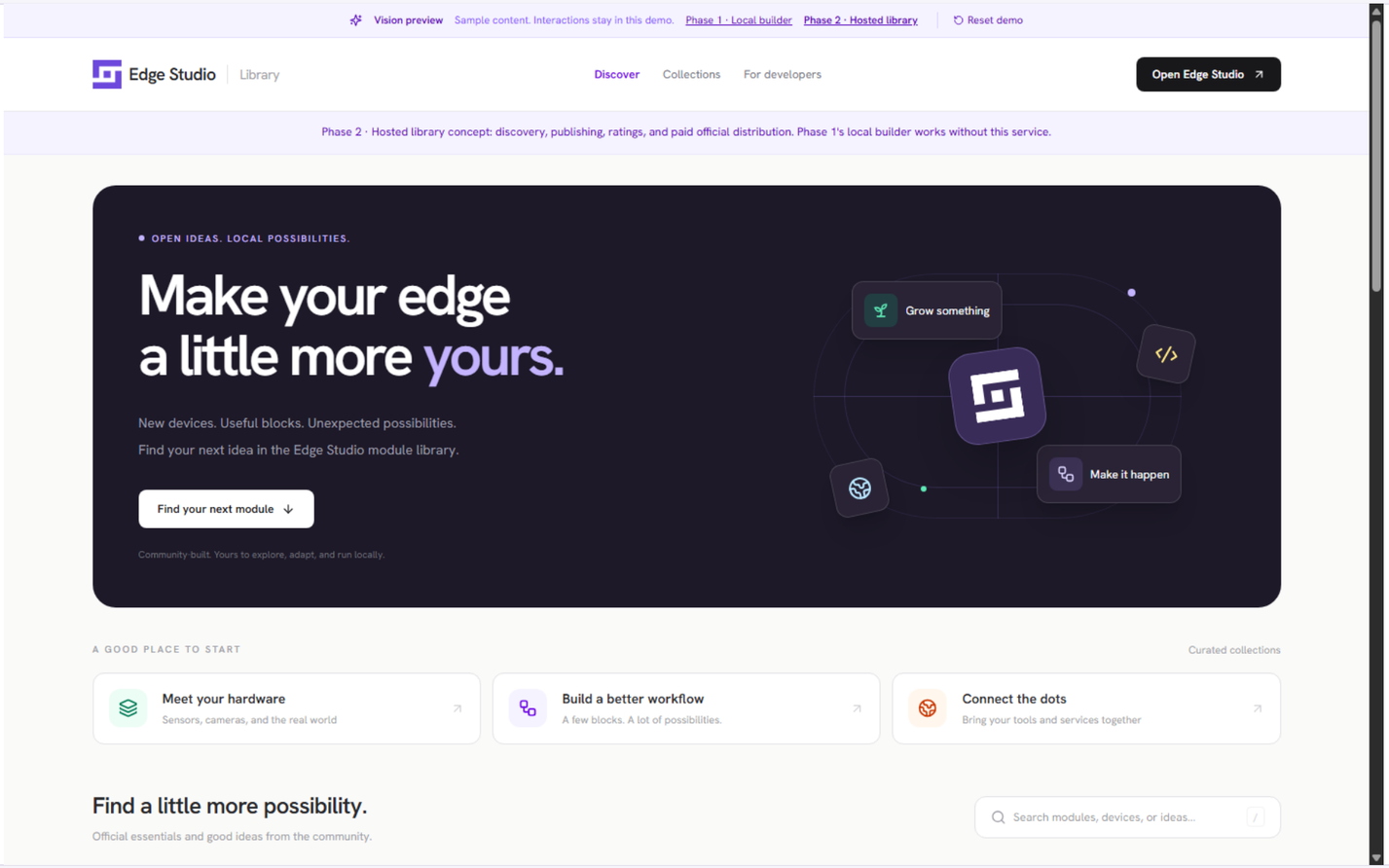
Keep three artifacts distinct

Source, installable packages and complete device backups serve different purposes.

Demo source/package JSON snapshots round-trip locally; production package formats are still to be defined.

11 / Discover a module in the hosted library

PHASE 2 · DISCOVER



Captured from the demo frontend; explanatory text is editable.

Focus: A public ecosystem around local packages

Find capabilities

Search by module and contribution type rather than waiting for a core release.

Understand the publisher

Publisher profiles, module ratings, aggregate builder ratings and downloads provide context.

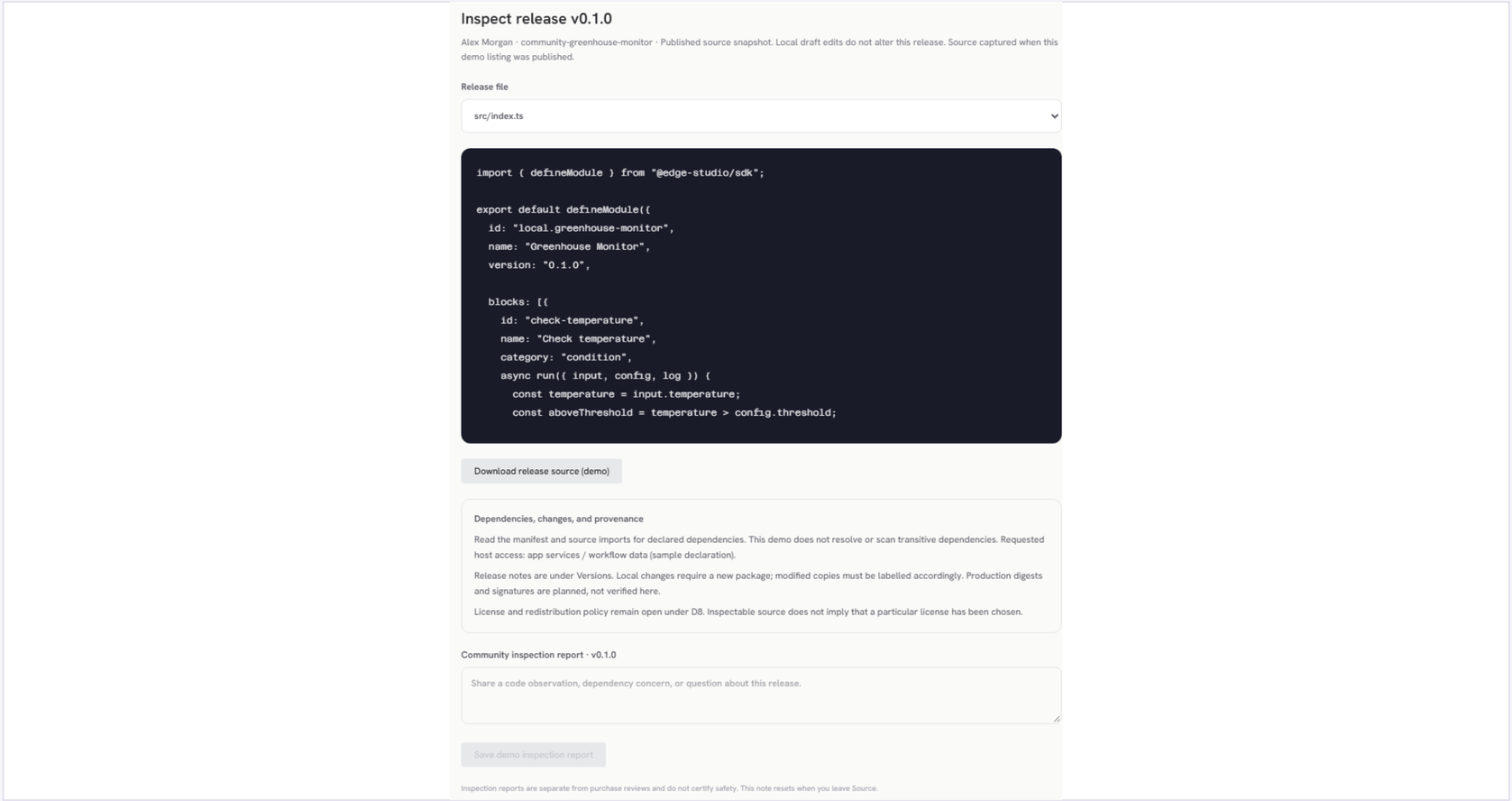
Retain operator choice

Inspect a release and its requested access before adding it to the device.

Ratings and download counts describe popularity and experience; they do not certify safety.

12 / Inspect the exact release source

PHASE 2 · INSPECT BEFORE INSTALLING



Captured from the demo frontend; explanatory text is editable.

Focus: Published source and community inspection

Inspect a versioned snapshot

The published release remains unchanged when an author later edits a local draft.

Review dependencies and access

Read the manifest, source and release notes; provenance must match the installed artifact.

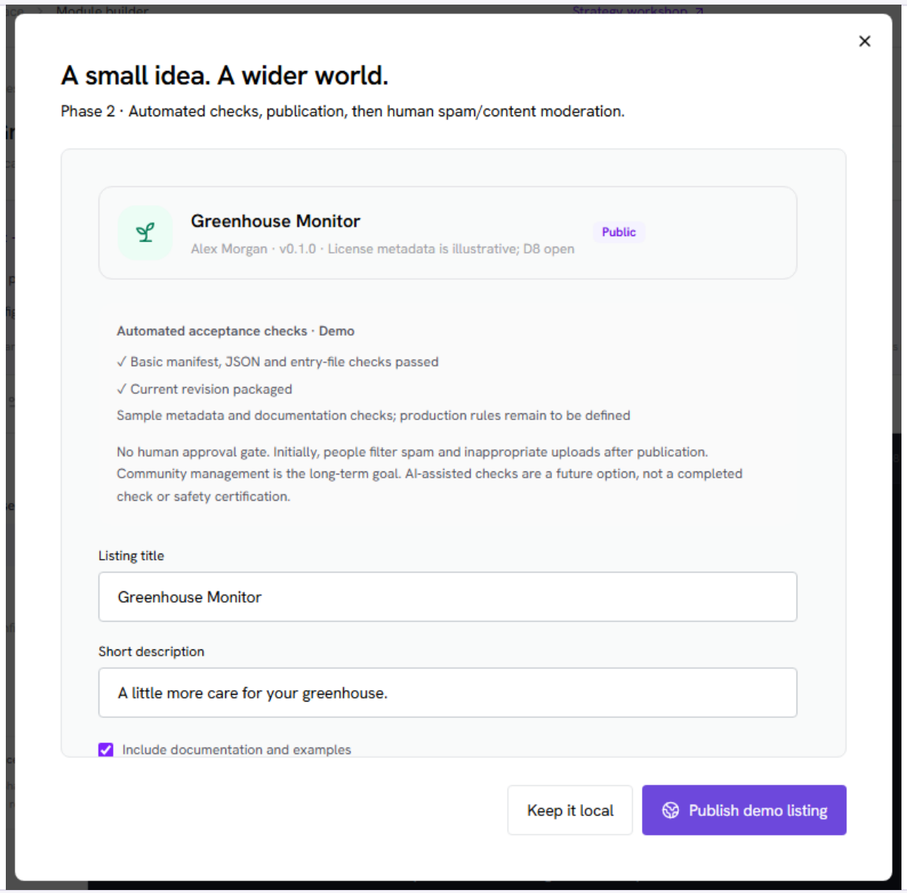
Separate the signals

Inspection reports are distinct from ratings and verified-purchase reviews.

Seeded source is illustrative; this newly published example captures the actual demo draft text.

13 / Publish after automated acceptance checks

PHASE 2 · PUBLISH



Captured from the demo frontend; explanatory text is editable.

Focus: No human approval gate

Package first

Publication uses a prepared revision plus metadata and documentation.

Run acceptance checks

Automated checks precede publication; exact production rules still need definition.

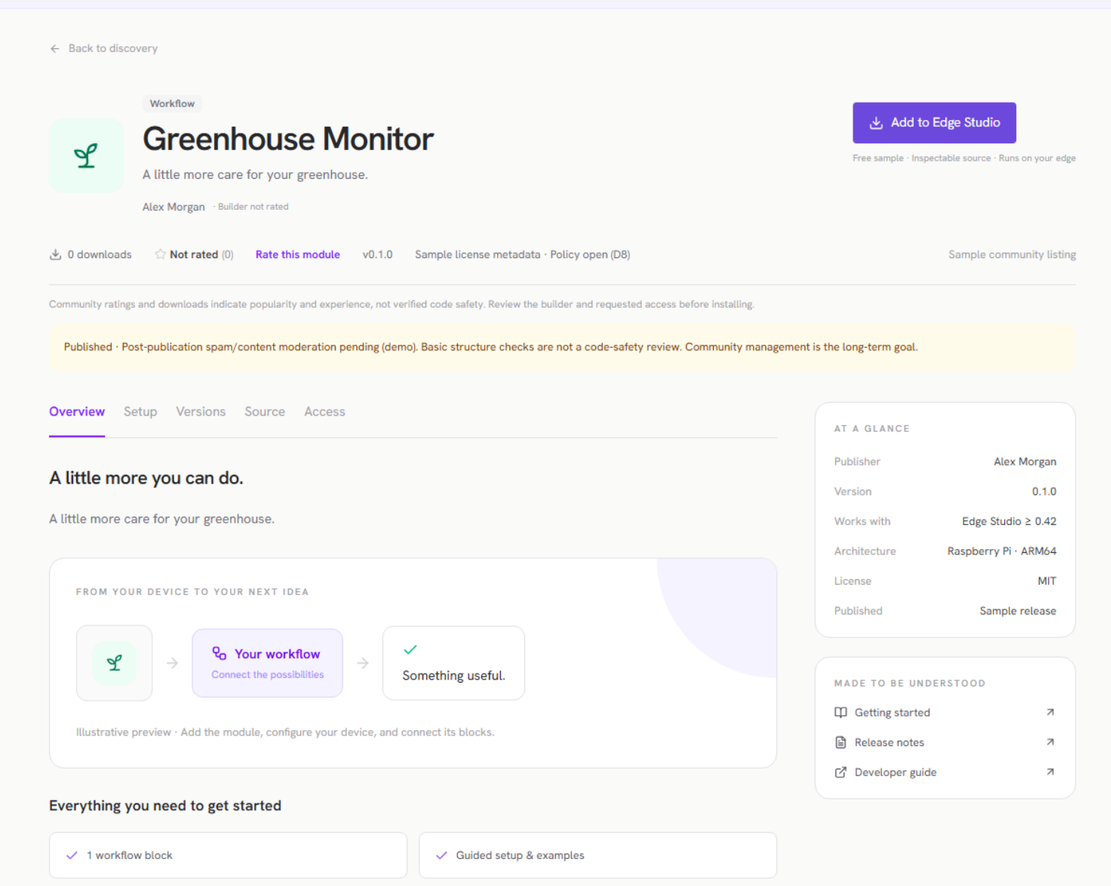
Acknowledge the contents

The author confirms their right to share; this is not a manual reviewer approval.

AI-assisted checks are an option to evaluate, not an implemented safety guarantee.

14 / Community feedback after publication

PHASE 2 · MODERATE AND LEARN



Captured from the demo frontend; explanatory text is editable.

Focus: Immediate listing, later content moderation

Initial rollout

Human moderation after publication filters spam and inappropriate uploads.

Long-term operating model

Community-managed inspection, reports and moderation supported by automation.

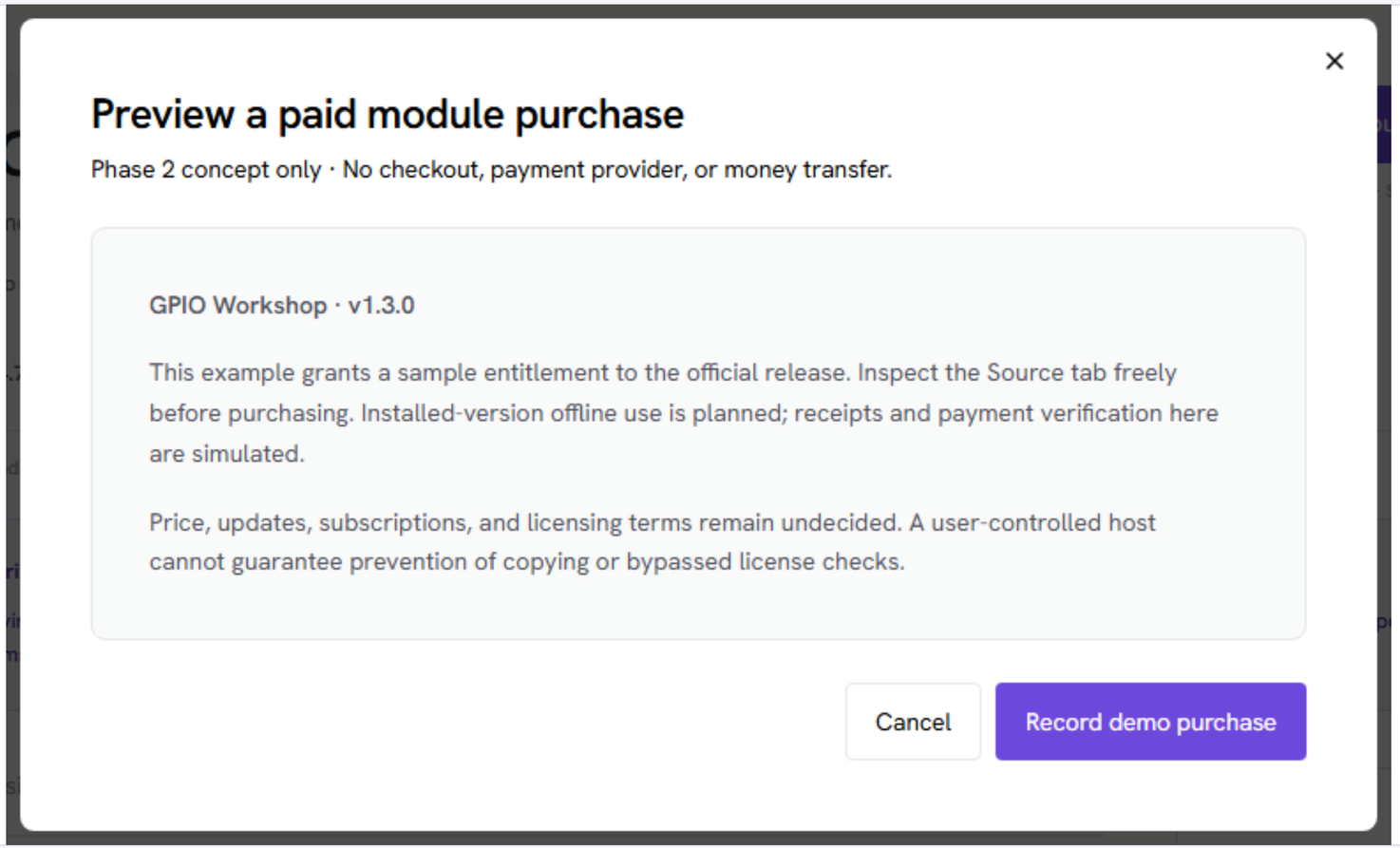
Separate service ownership

The web library is maintained separately from community module creation.

Named moderation owners, removal rules and transition criteria still need agreement.

15 / Paid official distribution, inspectable source

PHASE 2 · COMMERCIAL OPTION



Captured from the demo frontend; explanatory text is editable.

Focus: Entitlements without strong copy protection

Inspect before purchasing

Source remains inspectable; publisher identity and release provenance matter.

Use installed versions offline

The direction supports local entitlements without continuous license-server access.

Commerce is part of Phase 2

Paid distribution is required. Pricing, provider, subscription and redistribution/fork policies still need definition.

This is a simulated purchase. No money, checkout service or verified receipt is involved.

Delivery: local value first, hosted services second

The phase boundary is agreed. The public service is not a prerequisite for local use.

PHASE 1

M1 → M2 → M3

Lifecycle proof
Default-capability parity
Independent local authoring proof

PHASE 2

M4

Hosted catalog and publishing
Source inspection and reputation
Moderation and required commerce

The Phase 1 demo library is a lifecycle-test fixture, not the production publishing service.

How to read the development estimates

Bottom-up planning ranges based on scope; low confidence until the first lifecycle slice works.

CAPACITY

Two full-time engineers

One backend/runtime/hardware focus; one frontend/tools focus. Part-time QA/hardware and infrastructure support available.

MEASUREMENT

Effort, then elapsed time

One engineer-week is five focused workdays. Raw elapsed weeks = effort ÷ 2. Scheduling envelopes add about 20% for interruptions/integration.

INCLUDED

Delivery, not just code

Engineering review, automated tests, docs, integration and representative Pi acceptance are included in effort. The current frontend demo is reused.

EXCLUDED

Separate budgets and scope

No fully featured IDE, in-app AI harness, module CLI, arbitrary custom UI, universal hardware qualification or managed automatic rollback. Ongoing library operations are separate.

No start date, staffing commitment or budget is implied. Re-estimate after M1.

Phase 1 estimates: prove local extensibility

Engineering effort includes testing and integration; elapsed ranges assume two dedicated engineers.

Milestone / enabling work	Engineer-weeks	Raw elapsed	Acceptance evidence
App-wide backup/restore prerequisite	**6–10**	**3–5 weeks**	Export managed state + logs; restore on a test device.
M1 · Contracts + lifecycle slice	**8–12**	**4–6 weeks**	HTTP/Fetch removal, real demo download, activation and restart.
M2 · Default capability parity	**16–24**	**8–12 weeks**	Hardware/devices/blocks extracted; Pi parity and missing states.
M3 · Local authoring proof	**10–16**	**5–8 weeks**	New sensor/block from another author; package and install elsewhere.

Phase 1 total: 40–62 engineer-weeks · 20–31 raw weeks · about 24–38 scheduled weeks.

Phase 2 estimates: library and commerce

All three M4 work packages are included in the Phase 2 baseline.

M4 work package	Engineer-weeks	Raw elapsed	Acceptance evidence
M4a · Hosting + self-service publishing	**10–16**	**5–8 weeks**	Accounts, immutable artifacts, checks, listings and app delivery.
M4b · Reputation + community tools	**6–10**	**3–5 weeks**	Ratings, builder aggregates, counters, inspection and report flows.
M4c · Required commerce	**8–14**	**4–7 weeks**	Checkout, contributor payouts, refunds and offline entitlement.
M4 total · Includes commerce	**24–40**	**12–20 weeks**	Publish, inspect, buy, install and continue installed-version use offline.

Phase 2 total: 24–40 engineer-weeks · 12–20 raw weeks · about 15–24 scheduled weeks.

A planning envelope, not a promised launch date

Conservative sequential arithmetic; no unsupported overlap or AI productivity discount.

PHASE 1 · LOCAL BUILDER

40–62 engineer-weeks

Includes device export/restore prerequisite. Two engineers: about 24–38 scheduled weeks.

PHASE 2 · LIBRARY + COMMERCE

24–40 engineer-weeks

Includes checkout, contributor payouts and offline entitlements. Two engineers: about 15–24 scheduled weeks.

Both phases: 64–102 engineer-weeks · 32–51 raw weeks · about 39–62 scheduled weeks.

Validate the first slice before committing the full build

The next estimate should be grounded in implementation evidence, not a closer radar polygon.

PROVE

M1 acceptance

Remove → download → install inactive → activate → restart. Preserve existing configuration/history and expose missing-module states.

RESOLVE

Cross-app dependencies

App-wide support policy, export/restore coverage and recovery route, module/runtime contracts and named owners.

DECIDE

Phase 2 implementation policies

Commerce is required. Define provider/purchase terms, redistribution policy, publication checks and moderation ownership.

Proposed checkpoint: review M1 evidence, re-estimate M2–M4, then confirm the next funding/staffing step.